Part VIII

Parametric and Nonparametric Methods

Lecture Outline

Motivation: Continuation from Last Lecture

- Previously, we studied **Bayesian Decision Theory**, focusing on optimal decisions under uncertainty using $p_{Y|X}(Y \mid X) = \frac{p_{X|Y}(x|y)p_Y(y)}{p_X(x)}$. How do we estimate $p_{X|Y}(x \mid y), p_Y(y)$ from data if we do not know them?

Motivation: Continuation from Last Lecture

- Previously, we studied **Bayesian Decision Theory**, focusing on optimal decisions under uncertainty using $p_{Y|X}(Y \mid X) = \frac{p_{X|Y}(x|y)p_Y(y)}{p_X(x)}$. How do we estimate $p_{X|Y}(x \mid y), p_Y(y)$ from data if we do not know them?
- Even with a deterministic approach, if we have input data $x_1, \ldots, x_N$ and output data $y_1, \ldots, y_N$, how do we estimate a function which take a new test data x_t and predict its output y_t? ($y_t = f(x_t)$, what is f?).

Motivation: Continuation from Last Lecture

- **Two broad families of models**:
  - **Parametric**: Models have fixed, finite set of parameters θ (e.g., mean and variance of a Gaussian).
  - **Nonparametric**: The parameters' complexity of the models grows with data (e.g., histograms, kernel density, k-NN).

Motivation: Continuation from Last Lecture

- **Two broad families of models**:
  - **Parametric**: Models have fixed, finite set of parameters θ (e.g., mean and variance of a Gaussian).
  - **Nonparametric**: The parameters' complexity of the models grows with data (e.g., histograms, kernel density, k-NN).

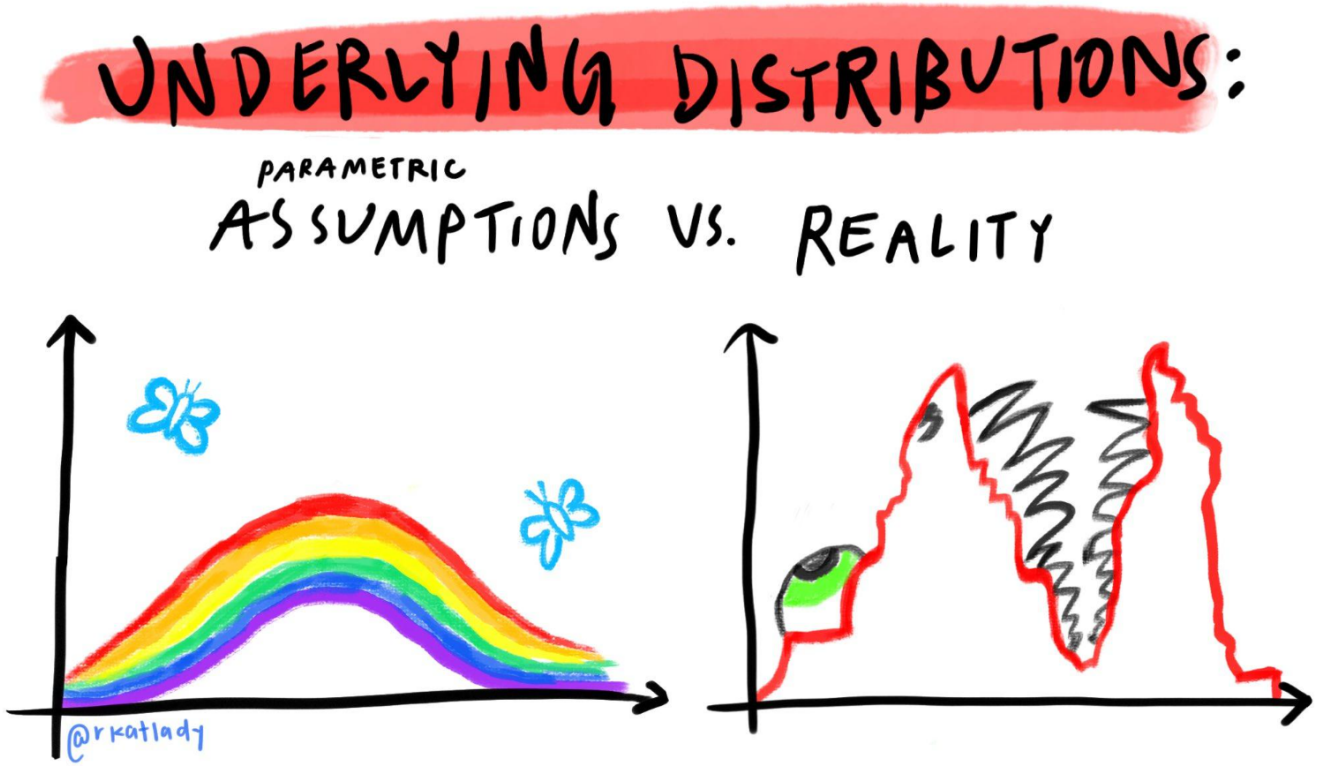
nonparametric does not mean
NONE-parametric (we still have parameters).

Parametric vs. Nonparametric: Key Differences

- **Parametric Methods**
  - Assume a specific form or function with fixed parameters θ (e.g., Gaussian distribution with mean and variance, linear model with slope and intercept).
  - A **fixed number of parameters** θ, independent of the dataset size.
  - Can be more robust to memory limitations.
  - Often faster once trained, but can be less flexible if the assumed form is poor.

Parametric vs. Nonparametric: Key Differences

- **Parametric Methods**
  - Assume a specific form or function with fixed parameters θ (e.g., Gaussian distribution with mean and variance, linear model with slope and intercept).
  - A **fixed number of parameters** θ, independent of the dataset size.
  - Can be more robust to memory limitations.
  - Often faster once trained, but can be less flexible if the assumed form is poor.



Parametric vs. Nonparametric: Key Differences

- **Nonparametric Methods**
  - Fewer assumptions about the model.
  - The **number of parameters (complexity of the model)** grow with the data size.
  - Can adapt more flexibly to data but can be slower at prediction time (often store all data).
  - Less robust to memory limitation (often store all data).

Parametric Methods: Overview

- Propose a family of distributions/models with a **fixed set of parameters** θ.
- **Estimate** θ from data using:
  - Maximum Likelihood Estimation (MLE)
  - Maximum A Posteriori (MAP)
  - The posterior's mean.
  - Least-Squares.
- Examples:
  - Linear/Logistic Regression (coefficients as parameters).
  - Gaussian distribution (mean, variance).

Parametric Methods: Overview

- Propose a family of distributions/models with a **fixed set of parameters** θ.
- **Estimate** θ from data using:
  - Maximum Likelihood Estimation (MLE)
  - Maximum A Posteriori (MAP)
  - The posterior's mean.
  - Least-Squares.
- Examples:
  - Linear/Logistic Regression (coefficients as parameters).
  - Gaussian distribution (mean, variance).
- **Hyperparameters** are the parameters we do not estimate. They are chosen by domain knowledge or other means.

Maximum Likelihood Estimation

Setup:

- Consider we have N *independent* samples $\{x_1, \ldots, x_N\}$ from a distribution p_X which we wish to estimate by $p_{X|\theta}(x \mid \theta)$ where θ are the fixed parameters of the model we choose.
- We treat our parameters as random variables too since we cannot exactly know their *true* values from data. We can only *estimate* them.

Maximum Likelihood Estimation

Setup:

- Consider we have N *independent* samples $\{x_1, \ldots, x_N\}$ from a distribution p_X which we wish to estimate by $p_{X|\theta}(x \mid \theta)$ where θ are the fixed parameters of the model we choose.
- We treat our parameters as random variables too since we cannot exactly know their *true* values from data. We can only *estimate* them.
- We assume a gaussian model $p_{X|\theta}(x \mid \theta) = \mathcal{N}(\mu, \sigma^2)$ with *unknown* variance σ^2 and unknown mean μ ($\theta = \begin{bmatrix} \mu \\ \sigma^2 \end{bmatrix}$):

$$p_{X|\theta}(x \mid \theta) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

- We want to fit a Gaussian distribution for the data by estimating the parameters ($\theta = \begin{bmatrix} \mu \\ \sigma^2 \end{bmatrix}$).

Maximum Likelihood Estimation

Setup:

- Consider we have N *independent* samples $\{x_1, \ldots, x_N\}$ from a distribution p_X which we wish to estimate by $p_{X|\theta}(x \mid \theta)$ where θ are the fixed parameters of the model we choose.
- We treat our parameters as random variables too since we cannot exactly know their *true* values from data. We can only *estimate* them.
- We assume a gaussian model $p_{X|\theta}(x \mid \theta) = \mathcal{N}(\mu, \sigma^2)$ with *unknown* variance σ^2 and unknown mean μ ($\theta = \begin{bmatrix} \mu \\ \sigma^2 \end{bmatrix}$):

$$p_{X|\theta}(x \mid \theta) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

- We want to fit a Gaussian distribution for the data by estimating the parameters ($\theta = \begin{bmatrix} \mu \\ \sigma^2 \end{bmatrix}$).
- Since $x_1, \ldots, x_N$ are independent and identically distributed (i.i.d) samples from p_X,

$$p_{X_1,\ldots,X_N|\theta}(x_1, \ldots, x_i \mid \theta) = p_{X|\theta}(x_1 \mid \theta) p_{X|\theta}(x_2 \mid \theta) \cdots p_{X|\theta}(x_N \mid \theta) = \prod_{n=1}^{N} p_{X|\theta}(x_n \mid \theta)$$

Maximum Likelihood Estimation

Setup:

- Consider we have N *independent* samples $\{x_1, \ldots, x_N\}$ from a distribution p_X which we wish to estimate by $p_{X|\theta}(x \mid \theta)$ where θ are the fixed parameters of the model we choose.
- We treat our parameters as random variables too since we cannot exactly know their *true* values from data. We can only *estimate* them.
- We assume a gaussian model $p_{X|\theta}(x \mid \theta) = \mathcal{N}(\mu, \sigma^2)$ with *unknown* variance σ^2 and unknown mean μ ($\theta = \begin{bmatrix} \mu \\ \sigma^2 \end{bmatrix}$):

$$p_{X|\theta}(x \mid \theta) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

- We want to fit a Gaussian distribution for the data by estimating the parameters ($\theta = \begin{bmatrix} \mu \\ \sigma^2 \end{bmatrix}$).
- The Maximum Likelihood Estimate (MLE) is the estimate

$$\hat{\theta}_{\text{MLE}} = \underset{\theta \in \Theta}{\text{argmax}} \prod_{n=1}^{N} p_{X|\theta}(x_n \mid \theta)$$

Maximum Likelihood Estimation

- Optimizing the logarithm of the likelihood (*log-likelihood*) is usually easier.
- For the Gaussian case with known variance

$$\ell(\theta) := \ln\left(\prod_{n=1}^{N} p_{X|\theta}(x_n \mid \theta)\right) = \sum_{n=1}^{N} -\frac{1}{2}\ln(2\pi\sigma^2) - \frac{(x_n-\mu)^2}{2\sigma^2}$$

Maximum Likelihood Estimation

- Optimizing the logarithm of the likelihood (*log-likelihood*) is usually easier.
- For the Gaussian case with known variance

$$\ell(\theta) := \ln\left(\prod_{n=1}^{N} p_{X|\theta}(x_n \mid \theta)\right) = \sum_{n=1}^{N} -\frac{1}{2}\ln(2\pi\sigma^2) - \frac{(x_n-\mu)^2}{2\sigma^2}$$

- The MLE estimate for the Gaussian case with known variance is

$$\hat{\theta}_{\text{MLE}} = \underset{\theta\in\mathbb{R}\times\mathbb{R}_{>0}}{\text{argmax}}\ \ell(\theta) = \begin{bmatrix}\hat{\mu}_{\text{MLE}} \\ \hat{\sigma}^2_{MLE}\end{bmatrix} = \begin{bmatrix}\frac{1}{N}\sum_{n=1}^{N} x_n \quad \text{(samples' average).} \\ \frac{1}{N}\sum_{n=1}^{N}(x_n-\hat{\mu}_{\text{MLE}})^2 \quad \text{(samples' variance)}\end{bmatrix}$$

Maximum Likelihood Estimation

Note that the maximum likelihood estimator $\hat{\sigma}^2_{\mathrm{MLE}}$ for the variance σ^2 is a biased estimator

$$\mathbb{E}\left[\hat{\sigma}^2_{\mathrm{MLE}}\right] = \frac{N-1}{N}\sigma^2.$$

Maximum Likelihood Estimation

Note that the maximum likelihood estimator $\hat{\sigma}^2_{\mathrm{MLE}}$ for the variance σ^2 is a biased estimator

$$\mathbb{E}\left[\hat{\sigma}^2_{\mathrm{MLE}}\right] = \frac{N-1}{N}\sigma^2.$$

We can correct it simply by

$$\hat{\sigma}^2_{\mathrm{UMLE}} = \frac{N}{N-1}\hat{\sigma}^2_{\mathrm{MLE}} = \frac{1}{N-1}\sum_{n=1}^{N}(x_n - \hat{\mu}_{\mathrm{MLE}})^2$$

Maximum A Posteriori (MAP) Estimation

- If we have a prior over our parameters, then we can obtain the estimate maximizing the *posterior*

$$\hat{\theta}_{\text{MAP}} = \operatorname*{argmax}_{\theta \in \Theta} \prod_{n=1}^{N} p_{\theta|X}(\theta \mid x_n) = \operatorname*{argmax}_{\theta \in \Theta} \prod_{n=1}^{N} p_{X|\theta}(x_n \mid \theta) p_{\theta}(\theta)$$

Maximum A Posteriori (MAP) Estimation

- If we have a prior over our parameters, then we can obtain the estimate maximizing the *posterior*

$$\hat{\theta}_{\text{MAP}} = \underset{\theta \in \Theta}{\text{argmax}} \prod_{n=1}^{N} p_{\theta|X}(\theta \mid x_n) = \underset{\theta \in \Theta}{\text{argmax}} \prod_{n=1}^{N} p_{X|\theta}(x_n \mid \theta) p_{\theta}(\theta)$$

- For the Gaussian case with *known* variance σ^2 but *unknown* mean μ ($\theta = \mu$) with a Gaussian prior on the mean $p_\theta(\theta) = p_\mu(\mu) = \frac{1}{2\sqrt{\pi\sigma_p^2}} \exp\left(\frac{-(\mu-\mu_p)^2}{2\sigma_p^2}\right)$, the MAP estimate is

$$\hat{\theta}_{\text{MAP}} = \hat{\mu}_{\text{MAP}} = \frac{N\sigma_p^2}{N\sigma_p^2 + \sigma^2} \frac{1}{N} \sum_{n=1}^{N} x_n + \frac{\sigma^2}{N\sigma_p^2 + \sigma^2} \mu_p.$$

Classification with Bayes

- We revisit **Bayesian Decision Theory** from the previous lecture, where the **optimal classification rule** (under 0–1 loss) is:

$$a^*(x) \;=\; a_{\text{argmax}_{c \in \mathcal{C}}\; p_{X \mid C}(x \mid c) p_C(c),\; \mathcal{C}=\{1,\ldots,K\}}$$

Classification with Bayes

- We revisit **Bayesian Decision Theory** from the previous lecture, where the **optimal classification rule** (under 0–1 loss) is:

$$a^*(x) \;=\; a_{\text{argmax}_{c \in \mathcal{C}}\, p_{X|C}(x \mid c) p_C(c),\; \mathcal{C}=\{1,\ldots,K\}}$$

- If we have N input-output pairs $(x_1, c_1), \ldots, (x_N, c_N)$, how do we estimate $p_{X|C}$ and p_C (if we do not have prior knowledge about the classes)?

Classification with Bayes

- We revisit **Bayesian Decision Theory** from the previous lecture, where the **optimal classification rule** (under 0–1 loss) is:

$$a^*(x) \;=\; a_{\text{argmax}_{c \in \mathcal{C}}\; p_{X|C}(x \mid c) p_C(c),\; \mathcal{C}=\{1,\ldots,K\}}$$

- If we have N input-output pairs $(x_1, c_1), \ldots, (x_N, c_N)$, how do we estimate $p_{X|C}$ and p_C (if we do not have prior knowledge about the classes)?
- **Parametric**: assume a parametric form for the distribution and find θ_ℓ and θ_p in $p(X \mid C, \theta_\ell)$ and $p_C(c \mid \theta_p)$ using, for example, MLE or MAP if we have a prior over the parameters θ_ℓ and θ_p.

Classification with Bayes

- We revisit **Bayesian Decision Theory** from the previous lecture, where the **optimal classification rule** (under 0–1 loss) is:

$$a^*(x) \;=\; a_{\text{argmax}_{c\in\mathcal{C}}\, p_{X|C}(x\,|\,c)p_C(c),\ \mathcal{C}=\{1,\ldots,K\}}$$

- If we have N input-output pairs $(x_1, c_1), \ldots, (x_N, c_N)$, how do we estimate $p_{X|C}$ and p_C (if we do not have prior knowledge about the classes)?
- **Example (1D features/inputs):** If the inputs (features) $x_1, \ldots, x_N$ are scalars, then we can use the Gaussian MLE estimate for each class. (x is the average blood glucose levels and $c = 1$ for the class of diabetic patient, while $c = 2$ for the class of a non-diabetic patient.
- For each class $c \in \{1, \ldots, K\}$, we will have

$$\theta_\ell^c = [\mu_c, \sigma_c^2]^\top$$

$$p_{X|C}(x \mid c, \theta_\ell^c) = \frac{1}{\sqrt{2\pi\sigma_c^2}} \exp\left(\frac{-(x-\mu_c)^2}{2\sigma_c^2}\right)$$

$$\hat{\mu}_{c_{MLE}} = \frac{1}{N_c}\sum_{n:c_n=c} x_n, \quad \hat{\sigma}^2_{c_{MLE}} = \frac{1}{N_c}\sum_{n:c_n=c} (x_n - \hat{\mu}_{c_{MLE}})^2.$$

Classification with Bayes

- We revisit **Bayesian Decision Theory** from the previous lecture, where the **optimal classification rule** (under 0–1 loss) is:

$$a^*(x) \;=\; a_{\text{argmax}_{c\in\mathcal{C}}\; p_{X|C}(x\,|\,c)p_C(c),\; \mathcal{C}=\{1,\ldots,K\}}$$

- If we have N input-output pairs $(x_1, c_1), \ldots, (x_N, c_N)$, how do we estimate $p_{X|C}$ and p_C (if we do not have prior knowledge about the classes)?
- For the prior p_C, since it is discrete $C \in \{1, \ldots, K\}$, we use a discrete distribution (also known as the categorical distribution) with probabilities $\theta_p = [\theta_{p,1}, \ldots, \theta_{p,K}]^\top$

$$p_Y(c \mid \theta_p) = \theta_{p,c}, \quad \theta_{p,c} : \text{is a probability.}$$

The MLE estimate is simply

$$\hat{\theta}_{p,c_{\text{MLE}}} = \frac{N_c}{N} \quad (\#\text{ points with class } c \text{ in data}/\#\text{ all data points})$$

Classification with Bayes

- We revisit **Bayesian Decision Theory** from the previous lecture, where the **optimal classification rule** (under 0–1 loss) is:

$$a^*(x) \;=\; a_{\text{argmax}_{c\in\mathcal{C}}\, p_{X|C}(x\,|\,c)p_C(c),\ \mathcal{C}=\{1,\ldots,K\}}$$

- If we have N input-output pairs $(x_1, c_1), \ldots, (x_N, c_N)$, how do we estimate $p_{X|C}$ and p_C (if we do not have prior knowledge about the classes)?
- **Summary (1D features/input: for each class $c \in \{1, \ldots, C\}$**

$$p_{X|C}(x \mid c, \theta_\ell^c) = \frac{1}{\sqrt{2\pi\hat{\sigma}^2_{c_{\text{UMLE}}}}} \exp\left(\frac{-(x-\hat{\mu}_{c_{\text{MLE}}})^2}{2\hat{\sigma}^2_{c_{\text{UMLE}}}}\right)$$

$$\hat{\mu}_{c_{MLE}} = \frac{1}{N_c}\sum_{n:c_n=c} x_n, \quad \hat{\sigma}^2_{c_{UMLE}} = \frac{1}{N_c-1}\sum_{n:c_n=c}(x_n-\hat{\mu}_{c_{MLE}})^2.$$

$$p_C(c \mid \theta_p) = \hat{\theta}_{p,c_{\text{MLE}}},$$

$$\hat{\theta}_{p,c_{\text{MLE}}} = \frac{N_c}{N}$$

Multivariate Gaussian Case (d-Dimensions)

- For d-dimensional features $x \in \mathbb{R}^d$, use the **Multivariate Gaussian Distribution** for each class c:

$$p_{x|C}(x \mid c, \theta_\ell^c) = \frac{1}{(2\pi)^{d/2}|\Sigma_c|^{1/2}} \exp\left(-\frac{1}{2}(x-\mu_c)^\top \Sigma_c^{-1}(x-\mu_c)\right)$$

Example: x is a vector containing BMI and average glucose levels, $c = 1$ is for a diabetic patient class, $c = 2$ for a non-diabetic class.

Multivariate Gaussian Case (d-Dimensions)

- For d-dimensional features $x \in \mathbb{R}^d$, use the **Multivariate Gaussian Distribution** for each class c:

$$p_{x|C}(x \mid c, \theta_\ell^c) = \frac{1}{(2\pi)^{d/2}|\Sigma_c|^{1/2}} \exp\left(-\frac{1}{2}(x-\mu_c)^\top \Sigma_c^{-1}(x-\mu_c)\right)$$

- **MLE Estimates for Class c:**
  - Mean vector:

$$\hat{\mu}_{c_{\text{MLE}}} = \frac{1}{N_c} \sum_{n:c_n=c} x_n$$

  - Covariance matrix:

$$\hat{\Sigma}_{c_{\text{UMLE}}} = \frac{1}{N_c - 1} \sum_{n:c_n=c} (x_n - \hat{\mu}_{c_{\text{MLE}}})(x_n - \hat{\mu}_{c_{\text{MLE}}})^\top$$

Multivariate Gaussian Case (d-Dimensions)

- For d-dimensional features $x \in \mathbb{R}^d$, use the **Multivariate Gaussian Distribution** for each class c:

$$p_{x|C}(x \mid c, \theta_\ell^c) = \frac{1}{(2\pi)^{d/2}|\Sigma_c|^{1/2}} \exp\left(-\frac{1}{2}(x-\mu_c)^\top \Sigma_c^{-1}(x-\mu_c)\right)$$

- **MLE Estimates for Class c:**
  - Mean vector:

$$\hat{\mu}_{c_{\text{MLE}}} = \frac{1}{N_c} \sum_{n:c_n=c} x_n$$

  - Covariance matrix:

$$\hat{\Sigma}_{c_{\text{UMLE}}} = \frac{1}{N_c - 1} \sum_{n:c_n=c} (x_n - \hat{\mu}_{c_{\text{MLE}}})(x_n - \hat{\mu}_{c_{\text{MLE}}})^\top$$

  - Class prior:

$$\hat{\theta}_{p,c_{\text{MLE}}} = \frac{N_c}{N} \quad \text{(same as 1D case)}$$

Naive Bayes for d-Dimensions

- **Problem**: Full covariance matrix Σ_c has $O(d^2)$ parameters ($d(d+1)/2$). For high d, this is computationally expensive and data-hungry.

Naive Bayes for d-Dimensions

- **Problem**: Full covariance matrix Σ_c has $O(d^2)$ parameters ($d(d+1)/2$). For high d, this is computationally expensive and data-hungry.
- **Naive Bayes Assumption**: Assume features are **conditionally independent** given the class:

$$p_{X|C,\theta_\ell}(x \mid c, \theta_\ell) = \prod_{i=1}^{d} p_{X_i|C,\theta_{\ell,i}}(x_i \mid c, \theta_{\ell,i})$$

  - This forces the covariance matrix Σ_c to be diagonal (features uncorrelated).
  - Examples: Given that you have the flu, the event that you are sneezing and the event that you are coughing are independent.

Naive Bayes for d-Dimensions

- **Problem**: Full covariance matrix Σ_c has $O(d^2)$ parameters ($d(d+1)/2$). For high d, this is computationally expensive and data-hungry.
- **MLE with Naive Bayes**:
  - For each class c and feature i:

$$\hat{\mu}_{c,i_{\text{MLE}}} = \frac{1}{N_c} \sum_{n:c_n=c} x_{n,i}$$

$$\hat{\sigma}^2_{c,i_{\text{UMLE}}} = \frac{1}{N_c - 1} \sum_{n:c_n=c} (x_{n,i} - \hat{\mu}_{c,i_{\text{MLE}}})^2$$

  - Reduces parameters from $O(d^2)$ to $O(d)$ per class.

Naive Bayes for d-Dimensions

- **Problem**: Full covariance matrix Σ_c has $O(d^2)$ parameters ($d(d+1)/2$). For high d, this is computationally expensive and data-hungry.
- **Why Use Naive Bayes?**
  - Efficient for high-dimensional data (e.g., text classification).
  - Robust to overfitting, even with small datasets.
  - Trade-off: Sacrifices modeling feature correlations for simplicity.

Nonparametric Methods: Key Features

- **No fixed number of parameters**: the complexity can grow with N.
- Typically store or reference the training data for inference.
- Examples:
  - Histograms (fixed bin widths).
  - Kernel Density Estimation (KDE).
  - k-Nearest Neighbors (k-NN).
- **Hyperparameters** are the parameters we do not estimate. They are chosen by domain knowledge or other means.

Density Estimation: Histograms

One dimensional case:

- The oldest and most popular method is the *histogram* where the space is divided into equal-sized intervals named *bins*.
- Given $x_1, \ldots, x_N \in \mathbb{R}$, we estimate the distribution $p_X(x)$ as

$$\hat{p}_X(x) = \frac{\#\{x - h/2 \leq x_n \leq x + h/2\}}{Nh}, \ n \in \{1, \ldots, N\}.$$

  where h is the fixed interval size.

Density Estimation: Histograms

One dimensional case:

- Given $x_1, \ldots, x_N \in \mathbb{R}$, we estimate the distribution $p_X(x)$ as

$$\hat{p}_X(x) = \frac{\#\{x - h/2 \leq x_n \leq x + h/2\}}{Nh}, \quad n \in \{1, \ldots, N\}.$$

  where h is the fixed interval size.

- We can also write the estimate as

$$\hat{p}_X(x) = \frac{1}{Nh} \sum_{n=1}^{N} \mathrm{K}\left(\frac{x - x_n}{h}\right),$$

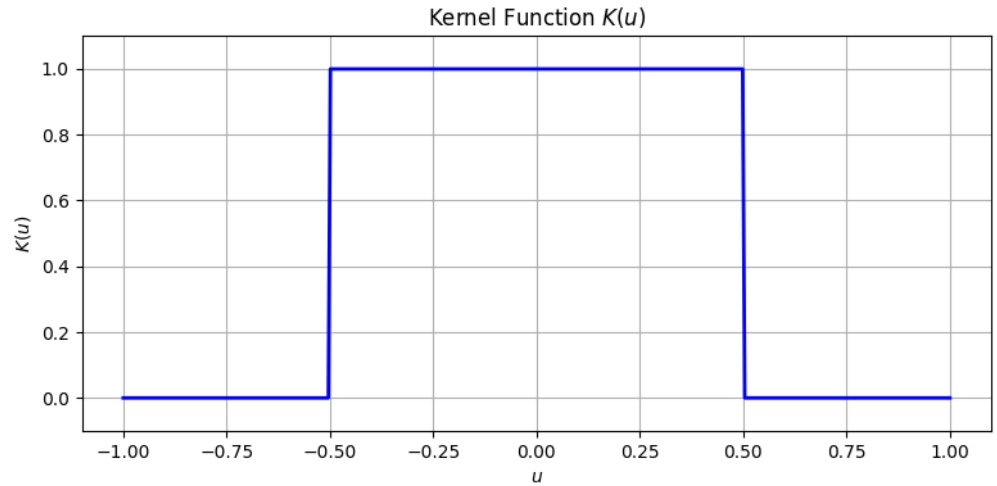
  with

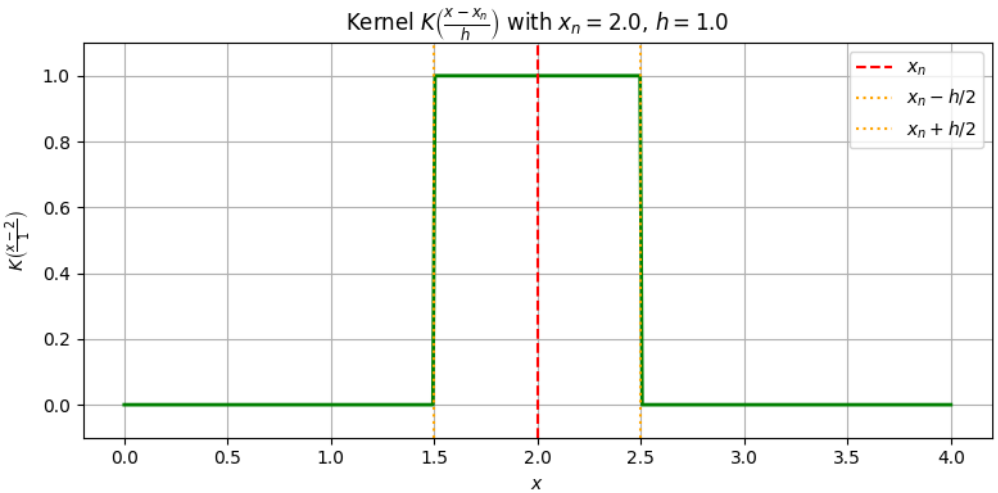
$$\mathrm{K}(u) = \begin{cases} 1, & |u| \leq 1/2, \\ 0, & \text{otherwise.} \end{cases}$$

Density Estimation: Histograms

One dimensional case:

- We can also write the estimate as $\hat{p}_X(x) = \frac{1}{Nh} \sum_{n=1}^{N} \mathrm{K}\left(\frac{x - x_n}{h}\right)$.

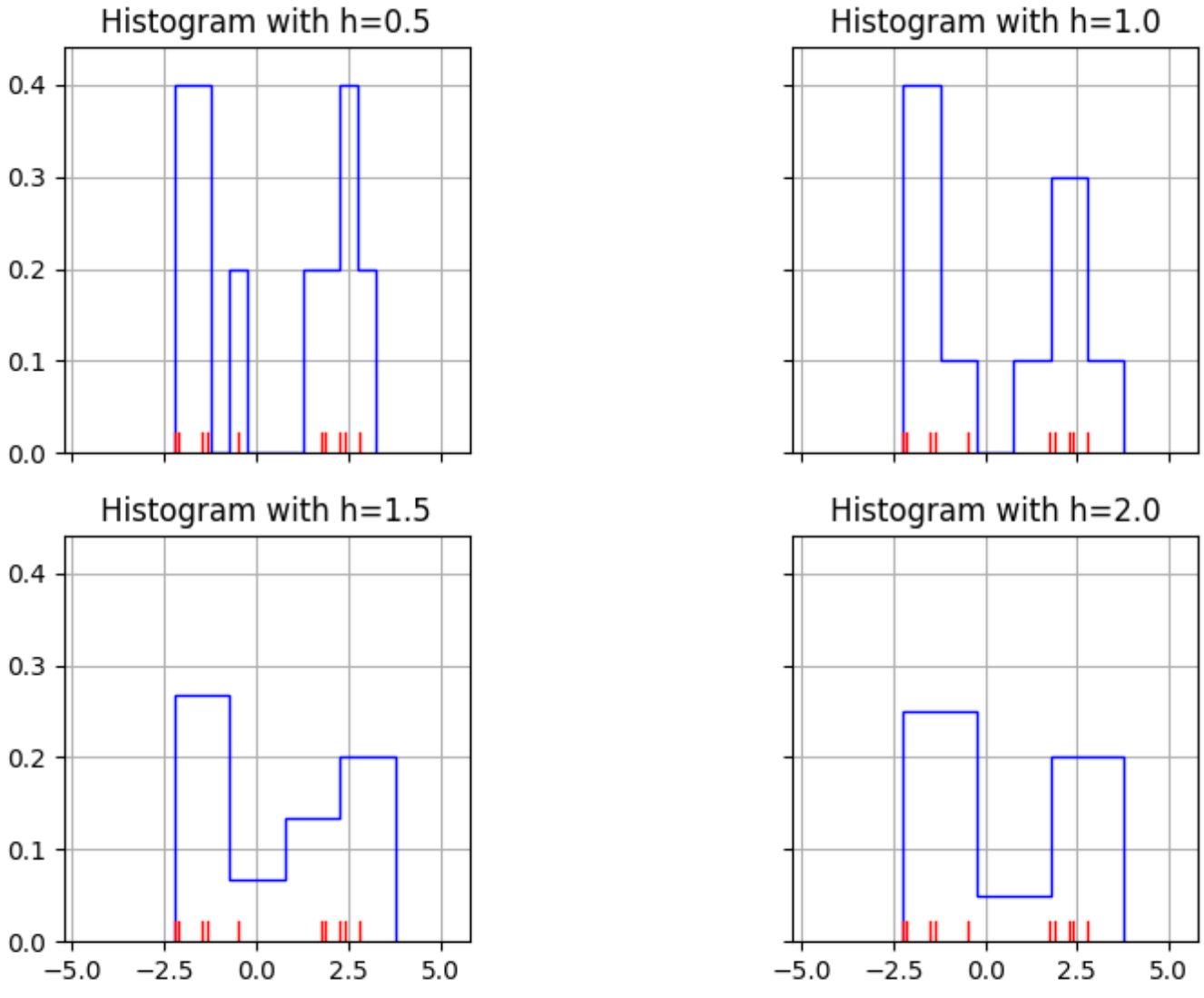




Density Estimation: Histograms

One dimensional case:

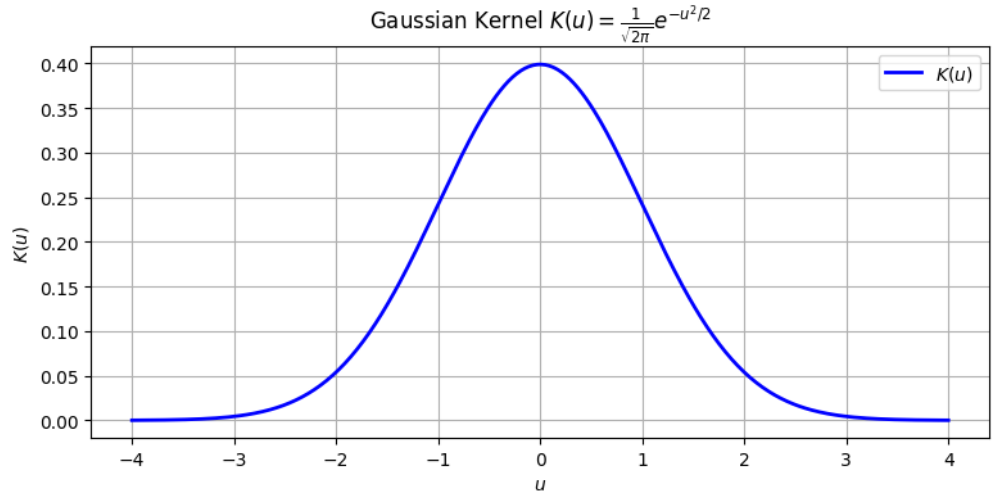
- Example:

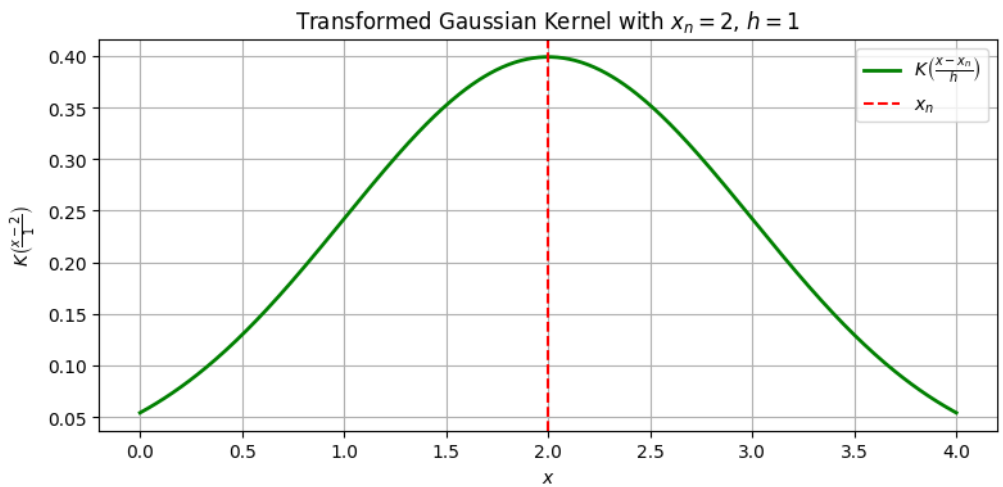


Density Estimation: Histograms

One dimensional case:

- To smooth the estimates, we can choose a gaussian kernel instead $\mathrm{K}(u) = \frac{1}{\sqrt{2\pi}} \exp(\frac{-u^2}{2})$

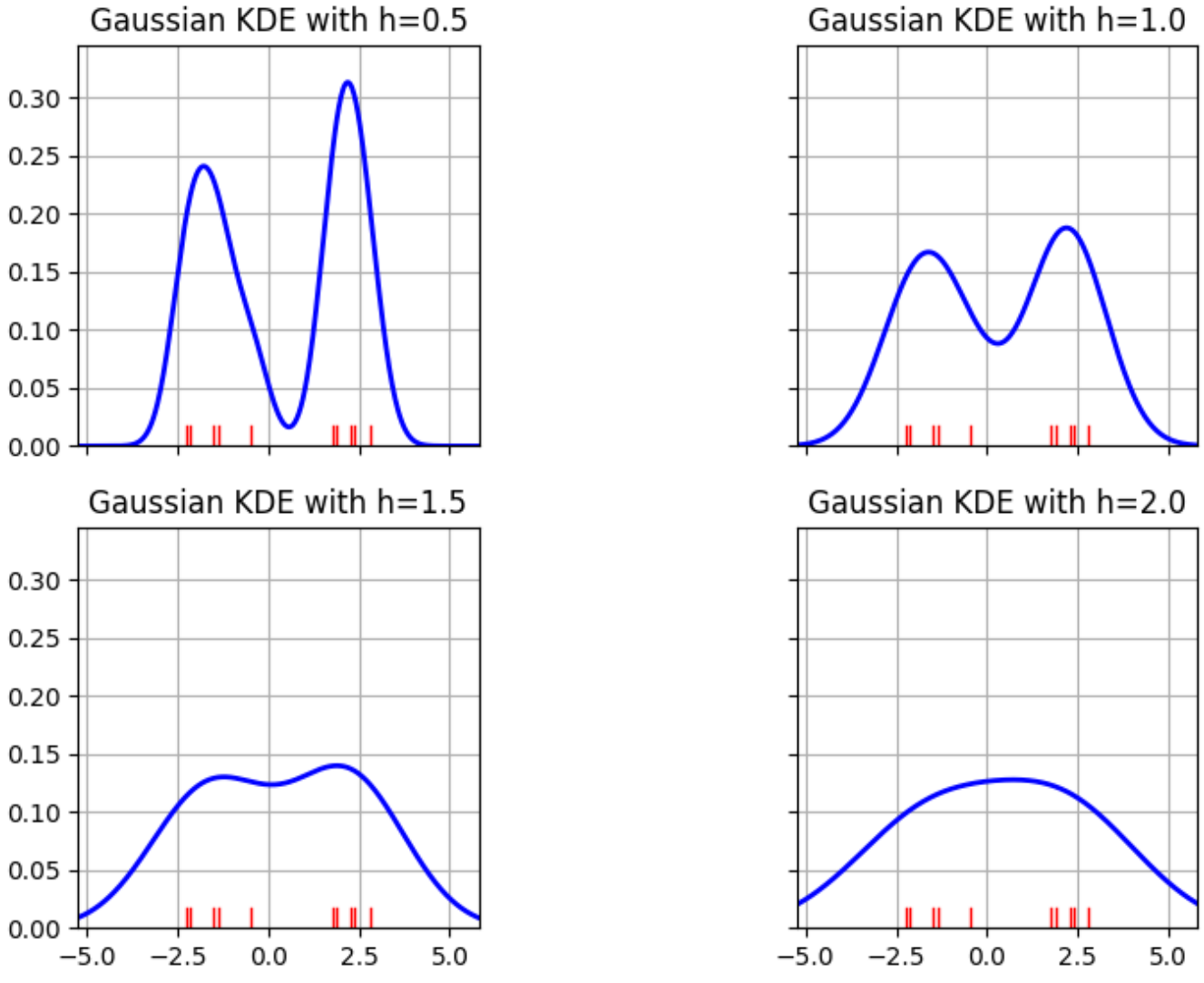




Density Estimation: Histograms

One dimensional case:

- Example with Gaussian kernel:



Density Estimation: k-Nearest Neighbors (k-NN)

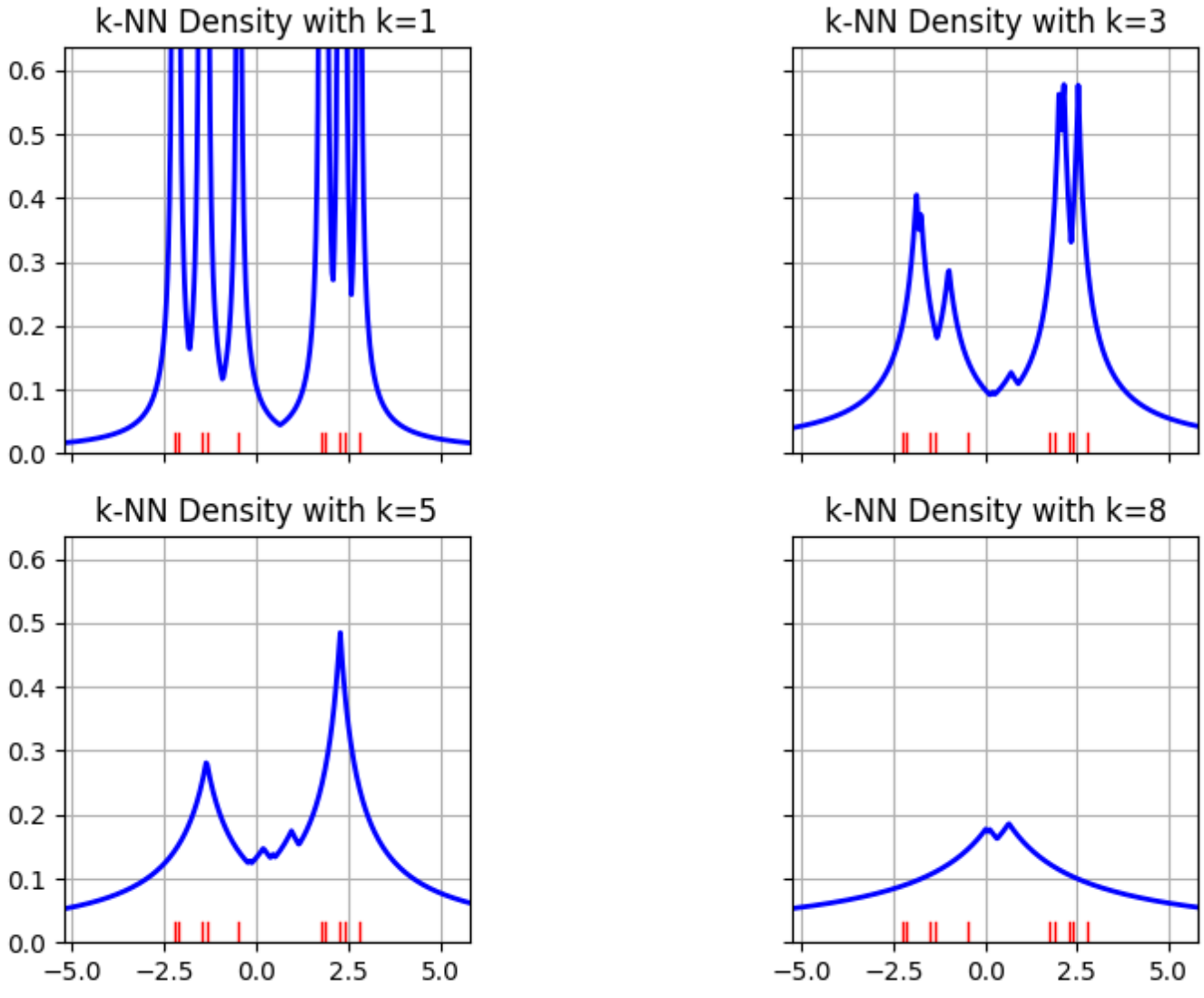
- **Basic Idea**: Instead of fixing the interval h, we fix the number of *neighbors* k and vary the interval

$$\hat{p}_X(x) = \frac{k}{2Nd_k(x)},$$

  where $d_k(x)$ is the distance to the k_{th} nearest data point x_n, $n \in \{1, \ldots, N\}$ to x.
- In k-NN, the "bin" around x is *implicitly* chosen so that it **just** contains k neighbors.
- If data are dense near x, the region is small (small "bin width").
- If data are sparse, the region is large.

Density Estimation: k-Nearest Neighbors (k-NN)

- Example:



Density Estimation: High-Dimensions

- In higher dimensions ($x \in \mathbb{R}^d$), we work with volumes instead of intervals:

$$\text{Histogram:} \quad \hat{p}_X(x) = \frac{1}{Nh^d} \sum_{n=1}^{N} \mathrm{K}\left(\frac{x - x_n}{h}\right),$$

$$k\text{-NN:} \quad \hat{p}_X(x) = \frac{k}{2NV_k(x)},$$

where $V_k(x)$ is the volume of the hypershpere centered at x.

Bayesian Classification with k-NN

- If we use K-NN to estimate

$$\hat{p}_X(x) = \frac{k}{2NV_k(x)},$$
$$\hat{p}_{X|C}(x \mid c) = \frac{k_c}{2N_c V_k(x)}$$
$$\hat{p}_C(c) = \frac{N_c}{N},$$

  then we have

$$\hat{p}_{C|X}(c \mid x) = \frac{\hat{p}_{X|C}(x \mid c)\hat{p}_C(c)}{\hat{p}_X(x)} = \frac{k_c \text{ (counts of neighbors from class c)}}{k \text{ (number of neighbors)}}.$$

Bayesian Classification with k-NN

- If we use K-NN to estimate

$$\hat{p}_X(x) = \frac{k}{2NV_k(x)},$$

$$\hat{p}_{X|C}(x \mid c) = \frac{k_c}{2N_c V_k(x)}$$

$$\hat{p}_C(c) = \frac{N_c}{N},$$

  then we have

$$\hat{p}_{C|X}(c \mid x) = \frac{\hat{p}_{X|C}(x \mid c)\hat{p}_C(c)}{\hat{p}_X(x)} = \frac{k_c \text{ (counts of neighbors from class c)}}{k \text{ (number of neighbors)}}.$$

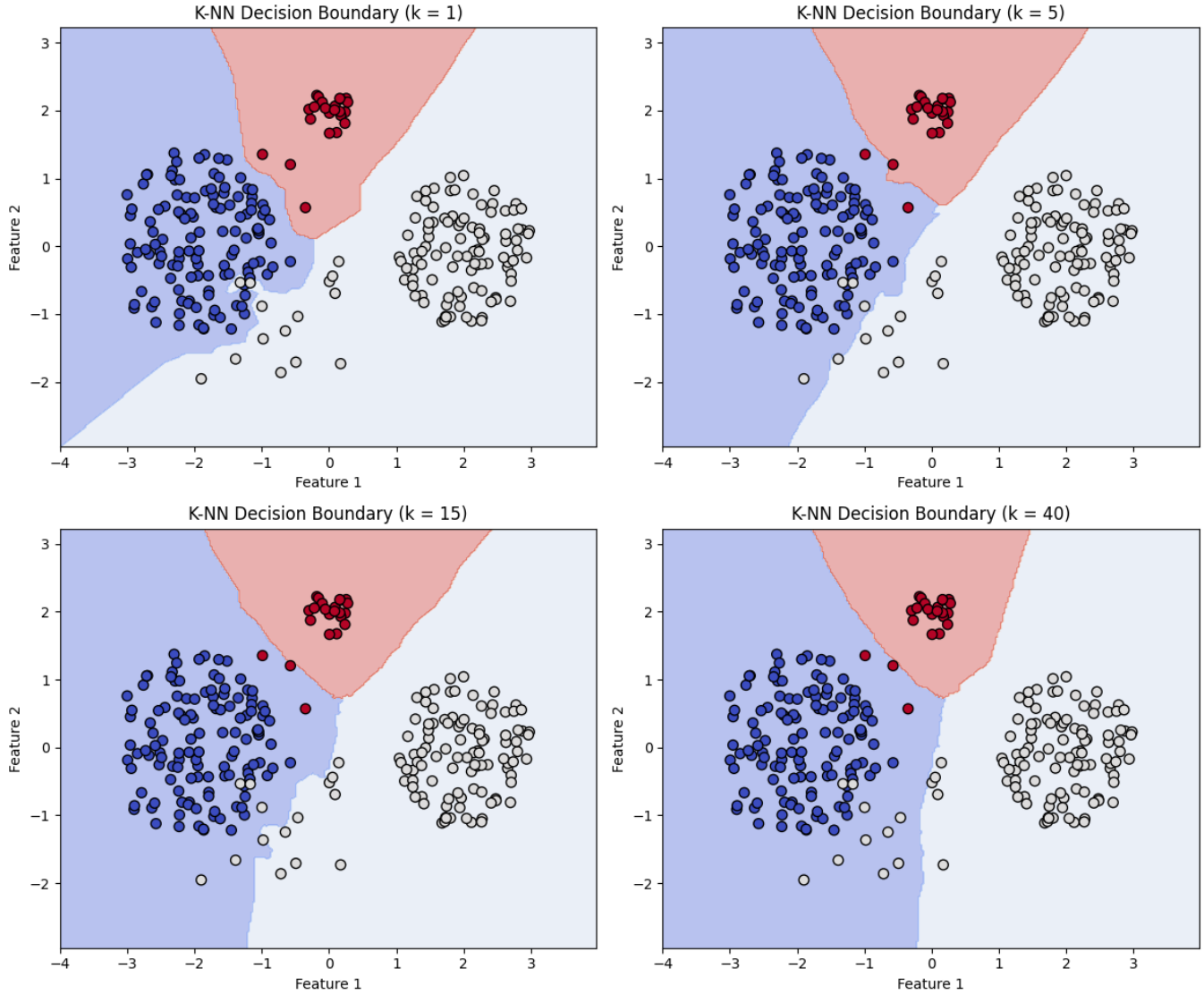
- So

$$a^*(x) = a_{\text{argmax}_{c \in \mathcal{C}} \frac{k_c}{k}}.$$

  In simple words, for a test point x which we want to classify, select the class c with the highest count among the k nearest neighbor to x.

Bayesian Classification with k-NN

- Decision Boundary



Summary

- **Parametric Methods**:
  - Fixed number of parameters (μ, σ^2) or regression coefficients.
  - MLE and MAP to estimate these parameters.
  - MAP connects to regularization via priors.
- **Nonparametric Methods**:
  - Complexity scales with data.
  - Histograms (fixed bin widths), Kernel methods, k-NN (variable width).
  - K-NN can estimate density (probabilistic) or do classification by majority vote.

Quantifying Classification Accuracy

- **Accuracy**: The proportion of correctly classified samples.

$$\text{Accuracy} = \frac{\#\text{ Correct Predictions}}{\#\text{ Total Predictions}}$$

- Error = 1-Accuracy.

Confusion Matrix

- A **confusion matrix** is a table that summarizes the performance of a classification model.
- For a binary classifier, it is structured as:

Actual \Predicted	Positive	Negative
Positive	#TP	#FN
Negative	#FP	#TN

- **Key Components:**
  - **TP (True Positives)**: Correctly predicted positive cases.
  - **FP (False Positives)**: Negative cases incorrectly predicted as positive.
  - **TN (True Negatives)**: Correctly predicted negative cases.
  - **FN (False Negatives)**: Positive cases incorrectly predicted as negative.
  - It can be also normalized by the total number of predictions.
- $$\text{Sensitivity} = \frac{\#\ \text{TP}}{\#\ \text{TP}+\#\text{FP}}$$
- $$\text{Specificity} = \frac{\#\ \text{TN}}{\#\ \text{TN}+\#\text{FN}}$$
- For multi-class classification, the matrix generalizes to a $C \times C$ table.